

Cobro con tarjeta tokenizada vía iframe

Flujo de 4 pasos para registrar al cliente, tokenizar la tarjeta y ejecutar el cobro con validación 3-D Secure.

API

DynamiCardPay

VERSIÓN

v2

DOCUMENTO

Rev. 1.0

FECHA

19 de agosto de 2026

Antes de empezar

El cobro con tarjeta se completa en cuatro pasos. El iframe de se usa exclusivamente para el registro de la tarjeta (tokenización); el resto del flujo se resuelve mediante llamadas a la API.

El flujo en cuatro pasos

Paso	Qué haces	Qué obtienes
1	Registras al cliente	customer_id (message.id)
2	Tokenizas la tarjeta en el iframe	token_id
3	Asocias la tarjeta al cliente	payment_method (message.id)
4	Generas el cobro	external_id y redirection_url (3DS)

Requisitos

Credencial	Para qué sirve	Dónde se usa
Public Key	Consumir el iframe de tokenización	Paso 2 (frontend)
KeyID	Identifica tu llave pública	Paso 2 (frontend)
API Key	Firmar las llamadas a la API	Pasos 1, 3 y 4 (backend)

Importante

La API Key nunca debe viajar al navegador. Las llamadas de los pasos 1, 3 y 4 se firman desde tu servidor con el algoritmo HMAC descrito en el anexo.

Cómo leer esta guía

Cada paso indica la URL, los headers, el body mínimo y la respuesta esperada. Los ejemplos de los pasos 1, 3 y 4 están tomados de Postman; el del paso 2 es HTML que corre en el navegador.

PASO 1 DE 4

1. Registrar al cliente

Este paso guarda la información personal del cliente. No registra la tarjeta: los datos que envías aquí sirven únicamente para crear el perfil del cliente.

1.1 • Endpoint

POST

<https://api.dynamicore.io/marketplace/apps/dynamicardpay/v2/customer/create>

1.2 • Headers

La petición debe incluir el header `Authorization`, que se construye según el anexo **Autorización (firma HMAC)**.

1.3 • Body

Este es el body mínimo requerido para guardar la información del cliente:

JSON

```
{
  "first_name": "Cliente",
  "last_name": "Apellido",
  "address_one": "Dirección No 123",
  "city": "CDMX",
  "state": "Ciudad de México",
  "zipcode": "00231",
  "email": "user@dynamicore.io",
  "country": "Mexico",
  "date_of_birth": "28/12",
  "last4ssn": "1234",
  "phone": "5500223355",
  "username": "userName"
}
```

1.4 · Respuesta

JSON

```
{
  "status": "success",
  "message": {
    "id": "74...f1",
    "company": "00",
    "first_name": "...",
    "last_name": "...",
    "address_one": "...",
    "city": "...",
    "state": "...",
    "zipcode": "...",
    "email": "...",
    "country": "...",
    "date_of_birth": "...",
    "last4ssn": "...",
    "phone": "...",
    "username": "...",
    "created_at": "...",
    "status": "Active",
    "commerce": "0000"
  }
}
```

Guarda este dato

message.id es el identificador del cliente. Lo necesitarás en el paso 3 para hacer el match con la tarjeta tokenizada, y en el paso 4 como customer_id.

PASO 2 DE 4

2. Tokenizar la tarjeta

La tokenización se realiza dentro del iframe que se proporciona. Al cargarlo, le pasas tu Public Key acompañada de su KeyID; los datos sensibles de la tarjeta nunca tocan tus servidores.

2.1 • Montar el iframe

HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <script src="https://d132t6c8viujz9.cloudfront.net"></script>
  <title> TOKENIZE</title>
  <script>
    const config = {
      targetIFrame: 'mainDiv',
      keyId: `TU_KEY_ID`,
      publicKey:
        `-----BEGIN PUBLIC KEY-----
        ...
        -----END PUBLIC KEY-----`,
    }
  </script>
</head>
<body>
  <div style="width: 100%; display: grid; place-items: center;" id="mainBody">
    <iframe id="mainDiv" style="border: none;"></iframe>
  </div>
</body>
</html>
```

2.2 • Qué ve el cliente

Una vez cargado, el iframe le pedirá los datos de la tarjeta:

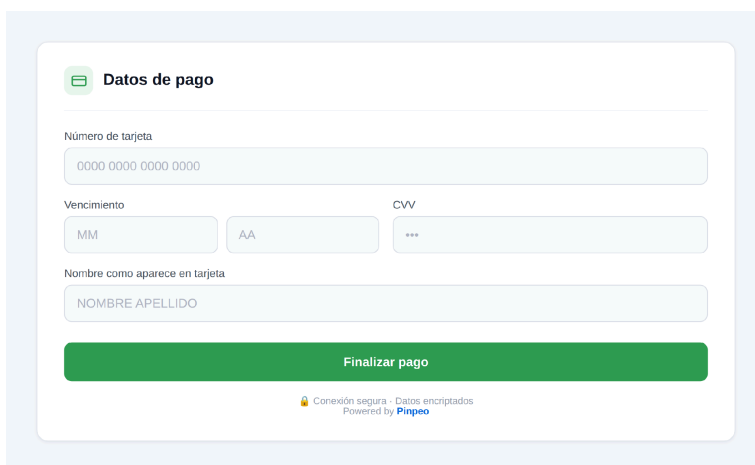
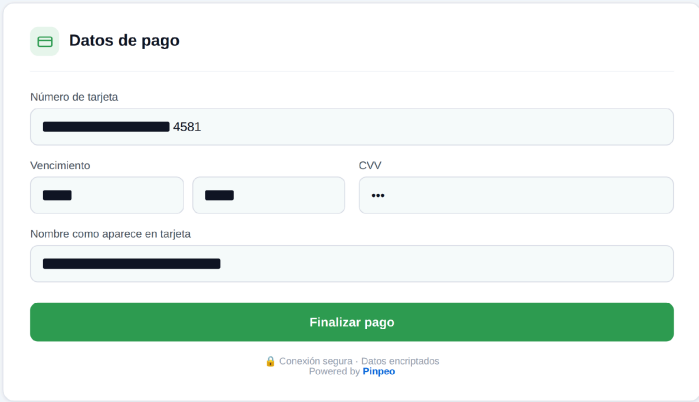
El formulario, titulado "Datos de pago", está diseñado para capturar información de una tarjeta de crédito. Incluye campos para el número de tarjeta (formato 0000 0000 0000 0000), el vencimiento (separado en campos MM y AA) y el CVV (formato ***). También hay un campo para el nombre tal como aparece en la tarjeta (formato NOMBRE APELLIDO). Un botón verde "Finalizar pago" se encuentra al final del formulario. En la parte inferior, se indica "Conexión segura · Datos encriptados" y "Powered by Pinpeo".

Figura 1. Formulario de tokenización, estado inicial.

La vista está pensada para que el cliente ingrese número de tarjeta, vencimiento (MM/AA), CVV y nombre del tarjetahabiente.

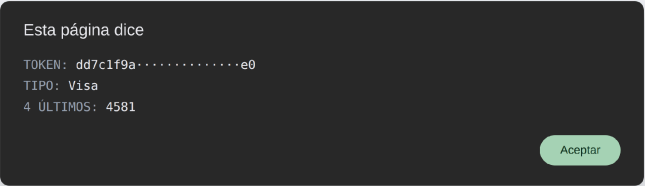


El formulario, titulado "Datos de pago", contiene los siguientes campos: "Número de tarjeta" con el valor "4581", "Vencimiento" con dos campos de fecha, "CVV" con tres asteriscos, y "Nombre como aparece en tarjeta" con un campo de texto. Debajo de los campos hay un botón verde que dice "Finalizar pago". En la parte inferior del formulario, se muestra un mensaje de seguridad: "Conexión segura · Datos encriptados" y "Powered by Pinpeo".

Figura 2. Formulario con los datos capturados.

2.3 · Recuperar el token

El servicio devuelve un id de token; es responsabilidad del integrador registrarlo para poder consumirlo al momento del pago. Por defecto el iframe lo entrega mediante un alert, pero puedes modificar ese evento según tu implementación.



La alerta muestra el siguiente texto: "Esta página dice", "TOKEN: dd7c1f9a.....e0", "TIPO: Visa", y "4 ULTIMOS: 4581". En la esquina inferior derecha hay un botón verde que dice "Aceptar".

Figura 3. Evento por defecto con el token generado.

Guarda este dato

El TOKEN devuelto es el token_id que usarás en el paso 3 para asociar la tarjeta al cliente.

PASO 3 DE 4

3. Relacionar cliente y tarjeta

Para asociar a un cliente con un método de pago —en este caso la tarjeta tokenizada— consume el siguiente endpoint.

3.1 • Endpoint

POST `https://api.dynamicore.io/dynamiccardpay/v2/card/assignToCustomer`

3.2 • Headers

Requiere el header `Authorization` firmado con tu API Key. Sin ella no tendrás acceso al endpoint.

3.3 • Body

Campo	Descripción	Origen
<code>customer_id</code>	Identificador del cliente	<code>message.id</code> del paso 1
<code>token_id</code>	Token de la tarjeta	Token generado en el paso 2

JSON

```
{
  "customer_id": "Id del cliente (message.id)",
  "token_id": "Token generado con el iframe"
}
```

3.4 • Respuesta

JSON

```
{
  "status": "success",
  "message": {
    "id": "682...11",
    "token": "dd...0",
    "default": "1",
    "client": "30...3",
    "created_at": "2026..."
  }
}
```

Guarda este dato

El `message.id` de esta respuesta es el método de pago. Se envía como `payment_method` en el paso 4 para realizar el cobro.

PASO 4 DE 4

4. Generar el cobro

Con el cliente registrado y la tarjeta asociada, ya puedes ejecutar el cargo.

NOTAS:

*1 El código de seguridad o SC es agregado cuando el cvv es dinámico esto aplica dependiendo el emisor. Usualmente el código viene en la tarjeta o en la aplicación.

2 Existen métodos que quitan el paso 1, 2 y 3 llamado cargo directo sin embargo esto depende de la integración a realizar. La forma segura es que se guarde mediante tokenización. En dado caso de hacer una integración mediante cargo directo es necesario pedir la guía de integración "Cobro con tarjeta - Cargo directo". También se debe tomar en cuenta que la información del cliente como de sus datos transaccionales pasan a ser responsabilidad del integrador.

4.1 • Endpoint

POST <https://api.dynamicore.io/dynamicardpay/v2/transactions/ccTransaction>

4.2 • Headers

Requiere el header `Authorization` firmado con tu API Key.

4.3 • Body

Campo	Descripción	Origen
payment_method	Método de pago	message.id del paso 3
customer_id	Identificador del cliente	message.id del paso 1
amount	Monto a cobrar	Tu comercio
sc	security code *Código de seguridad de la tarjeta CVV(Es opcional ponerlo siempre y cuando el emisor permita cvv estático)	Tarjetahabiente
accept_url	Redirección tras un pago aprobado	Tu comercio
cancel_url	Redirección tras un pago cancelado	Tu comercio

JSON

```
{
  "payment_method": "xxxxxxxx-xxx... (Método de pago del paso 3)",
  "customer_id": "Id del cliente (message.id)",
  "amount": 15,
  "sc": 123, //puede o no llevarlo*1
  "accept_url": "https://tu-tienda.com/pago/exito",
  "cancel_url": "https://tu-tienda.com/pago/cancelado"
}
```


4.4 · Respuesta

JSON

```
{
  "status": "success",
  "message": {
    "external_id": "trs_...",
    "date": "2026-08-17T22:50:33.554525+00:00",
    "client": "0d...",
    "message": "RD",
    "redirection_url": "https://..."
  }
}
```

4.5 · Validación 3-D Secure

El flujo continúa en el *challenge*: redirige al cliente a la URL de `redirection_url`, donde el banco emisor solicita la autorización del pago.

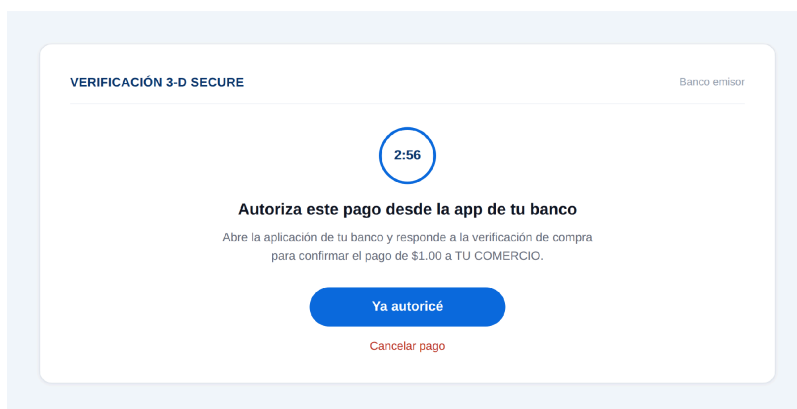


Figura 4. Pantalla de challenge 3-D Secure del banco emisor.

Una vez completado el challenge sabrás si el cobro fue aprobado o rechazado, y el cliente será redirigido a tu `accept_url` o `cancel_url` según corresponda.

ANEXO A

Autorización (firma HMAC)

Todas las llamadas a la API (pasos 1, 3 y 4) viajan con el header `Authorization`. El siguiente script de *pre-request* de Postman lo construye a partir del método, la URL y el body de la petición.

Sustituye tus credenciales

Antes de ejecutarlo, coloca tu `CLIENT_KEY` y el hash de tu secreto en las variables correspondientes. Este script debe vivir únicamente en tu backend o en tu entorno de pruebas.

Construcción del header

JAVASCRIPT

```
function getPath(url) {
  var pathRegex = /.+?\:\/\/\.\.+?(\/\.\.+?)(?:#|\?|$)/;
  var result = url.match(pathRegex);
  return result && result.length > 1 ? result[1] : '';
}

function getQueryString(url) {
  var arrSplit = url.split('?');
  return arrSplit.length > 1 ? url.substring(url.indexOf('?') + 1) : '';
}

var _requestData;
var _requestPath;

function getAuthHeader(httpMethod, requestUrl, _requestBody) {
  var CLIENT_KEY = '';
  var hash = '';
  var SECRET_KEY = CryptoJS.enc.Hex.stringify(CryptoJS.SHA512(hash));
  var AUTH_TYPE = 'DyamiCore';
  var requestBody = '';
  var requestPath = getPath(requestUrl);
  _requestPath = requestPath;
  var queryString = getQueryString(requestUrl);

  if (httpMethod == 'GET' || !_requestBody) {
    requestBody = '';
  } else {
    requestBody = JSON.stringify(JSON.parse(_requestBody));
  }

  var timestamp = new Date().getTime();
  var requestData = [timestamp, httpMethod, requestPath, queryString, requestBody].join("");
  requestData = requestData.trim();
  _requestData = requestData;

  var hmacDigest = CryptoJS.enc.Hex.stringify(CryptoJS.HmacSHA256(requestData, SECRET_KEY));
  var authHeader = AUTH_TYPE + ' ' + CLIENT_KEY + ':' + timestamp + ":" + hmacDigest;
  return authHeader;
}

postman.setEnvironmentVariable('hmacAuth', getAuthHeader(request['method'], request['url'], request['data']));
postman.setEnvironmentVariable('hmac_d', _requestData);
postman.setEnvironmentVariable('hmac_path', _requestPath);
```